

- El **SGBD** (o DBMS en inglés) es el intermediario entre usuarios y datos (ej: MySQL, PostgreSQL, Oracle, SQL Server).
- El **motor de base de datos** es la **capa de software inferior que interactúa directamente con el sistema operativo y el disco duro** (ej: InnoDB, MyISAM, RocksDB).
- **El nivel conceptual en ANSI/SPARC describe** El modelo lógico global de la base de datos
- La arquitectura ANSI/SPARC se compone de 3 niveles:
 - o Nivel externo: vistas de usuario.
 - o
- **La finalidad de la normalización es evitar** que los datos se repitan (**redundancia**), evitar errores al insertar/actualizar/borrar (**inconsistencia**), garantizar la integridad. y mejorar la estructura.
 - o 1FN -> valores atómicos.
 - o 2FN -> dependencias parciales. (nadie depende de media clave).
 - o 3FN -> Dependencias transitivas indirectas entre atributos no clave (nadie depende de otro no clave).
- DDL define estructura y DML manipula datos.
- Check valida reglas o condiciones sobre los datos.
- Definir correctamente los tipos de datos evita errores y optimiza almacenamiento.
- La restricción CHECK valida reglas o condiciones sobre los datos.
- Un registro. *(También llamado "tupla", representa una instancia específica y completa de la tabla).*
- Una tabla cumple BCFN si cada determinante es clave candidata.
- Una desnormalización controlada se usa para mejorar rendimiento.
- La integridad referencial garantiza que las FK apunten a valores

existentes.

- El diseño lógico consiste en Convertir el modelo E-R en tablas.
- WHERE filtra ANTES de GROUP BY.
- HAVING filtra GRUPOS.
- Para filtrar sólo los que sean NULL a un lado de una JOIN se usaría LEFT JOIN + WHERE NULL.
- La tabla intermedia en una relación Muchos a Muchos (N:M) no representa un objeto nuevo, sino la relación entre dos objetos existentes. La combinación de las dos Claves Foráneas (FK) es la identidad real de ese registro y es única en el mundo real.
- **Reglas Básicas de Transformación de ER a Relacional:**
 - o 1. entidad -> tabla
 - o 2. atributos
 - o 3. Id -> pk
 - o 4. Relaciones -> fk
 - o 5. N:m -> tabla intermedia
 - o Atributos multivalorados -> nueva tabla (NO ENUM)

ANSI/SPARC: . Su objetivo principal es separar las aplicaciones de los usuarios de los detalles físicos de almacenamiento, permitiendo ocultar la complejidad y facilitando la independencia de los datos:

- **Nivel Externo (o de Vista):** Es la capa más cercana al usuario.
- **Nivel Conceptual:** Es la vista global o comunitaria de la base de datos.
- **Nivel Interno (o Físico):** Es la capa más cercana al almacenamiento físico. Describe cómo se guardan realmente los datos en el disco (estructuras de almacenamiento, índices, compresión y rutas de acceso).

SQL

- **Comentarios:** -- comentario (con espacio), #comentario o /* comentario largo */.
- **Tipos comunes:** INT/BIGINT ; DECIMAL(0,0) ; VARCHAR(0) ; DATE/DATETIME/TIMESTAMP ; BOOLEAN (TINYINT(1) EN MARIA) ; ENUM('A', 'B'): Valores cerrados específicos.

DDL:

RESTRICCIONES DE INTEGRIDAD: Reglas de seguridad que protegen a la BD de errores estructurales o de información.

- **RESTRICC DE DOMINIO:** Aseguran **datos válidos** en cada columna.
 - o NOT NULL CHECK
- **RESTRICC DE ENTIDAD:** “Una PK nunca es NULL ni repetida “
 - o PK UNIQUE AUTO_INCREMENT
- **RESTRICC REFERENCIALES:** “Toda FK debe apuntar a un registro existente”
 - o FK: **coherencia lógica entre tablas.** Se señalan mediante:
 - **CONSTRAINT** fk_nombre_relacion
 - **FOREIGN KEY** (nombre_nuevaTabla_campo)
 - **REFERENCES** nombre_tablaDePK (nombre_campo)
 - o ON DELETE CASCADE: Si se borra un registro en una tabla de la que depende otra, se borran todos los registros asociados.
 - o ON UPDATE CASCADE: actualiza los registros vinculados si se modifica la PK de la tabla padre.

CREATE, ALTER Y DROP -> Create permite: añadir (ADD), modificar el tipo (MODIFY), renombrar (CHANGE) o borrar (DROP COLUMN) **columnas existentes.**

DML:

El DML se encarga de gestionar el ciclo de vida de un dato, y permite alterar la información interna de las tablas. Son las operaciones de escritura del acrónimo CRUD (Create, Update, Delete), las cuales se apoyan en la cláusula WHERE.

Operac b. -> INSERT INTO, UPDATE (modifica mediante SET y WHERE), DELETE (tmb con SET y WHERE).

DQL:

Abarca exclusivamente la extracción y lectura de datos sin modificarlos.

El comando SELECT es el núcleo del lenguaje DQL. Dominarlo significa poder transformar una masa de datos inhertes en información útil, comprensible y accionable.

La estructura básica de una consulta SELECT es:

SELECT	+	FROM	+	WHERE
(qué información		(de dónde		(condición
- columna)		-tabla)		-filas)

Cláusulas (bloque lógico de instrucciones) de una consulta SELECT:

- o **SELECT:** (Obligatoria) Define columnas, cálculos o funciones matemáticas .
- o **FROM:** (Obligatoria) Indica la tabla.
- o **JOIN:** (acompañada de ON) para vincular y cruzar datos.
- o **WHERE:** Filtra antes de cualquier agrupación.

- **Operadores:**

- **De comparación:** = != < >
 <= >=
- **Lógicos:** AND, OR, NOT e IS NULL.
- **Avanzados:**
 - **IN:** Para buscar coincidencias dentro de una lista
 - **BETWEEN:** Para buscar valores dentro de un rango.
 - **DISTINCT:** Para evitar duplicados.
 - **LIKE:** Para búsquedas por patrones de texto usando % (cualquier cantidad de caracteres) y _ (un solo carácter).
 - comodines %%%
 - que empiece por m -> "m%"
 - que acabe por m -> "%m"
 - que contenga la m -> "%m%"
- **GROUP BY:** Agrupa filas con los mismos valores.
- **HAVING:** **filtra después de agrupar.**
- **ORDER BY:** Ordena el bloque final de resultados.
- **LIMIT** (y su acompañante OFFSET): Recorta el resultado final.

Las funciones agregadas (COUNT ; SUM ; AVG ; MIN ; MAX) se pueden utilizar:

- **En el SELECT** (Para mostrar el resultado).
- **En el HAVING** (Para filtrar los grupos).
- **En el ORDER BY** (Para ordenar por el resumen).

PERO NUNCA EN EL WHERE.

- **INNER JOIN:** devuelve coincidencias.
 - **LEFT JOIN:** Devuelve la información de la tabla principal aunque no haya coincidencias *Detecta registros sin actividad.*
 - **RIGHT JOIN:** Lo contrario que left. *Detecta registros huérfanos*
 - **FULL JOIN:** Devuelve todas las filas de ambas tablas, uniendo cuando hay coincidencias y mostrando NULL cuando no las hay.
- Aunque PostgreSQL y SQL Server sí soportan FULL JOIN, MySQL no, y para hacerlo en este motor habría que hacer:**
LEFT JOIN (todo A + coincidencias de B)

RIGHT JOIN (todo B + coincidencias de A)
UNION para no duplicar lo que coincide en ambos.

Subconsultas (IN y EXISTS): Una subconsulta es una consulta anidada dentro de otra: Sirve para filtrar, calcular o incluso generar tablas intermedias

La sintaxis más común es encontrar la subconsulta dentro de una cláusula WHERE, pero también puede usarse en FROM (como tabla virtual) o incluso en SELECT (para calcular valores derivados).

TCL - Control de Transacciones

El Modelo ACID garantiza la fiabilidad de las transacciones.

- **A (Atomicidad):** "Todo o nada".
- **C (Consistencia):** La base de datos siempre pasa de un estado válido a otro estado válido.
- **I (Isolation/Aislamiento):** Si hay varias transacciones a la vez, cada una se comporta como si estuviera sola.
- **D (Durabilidad):** Tras confirmar (COMMIT), la transacción persiste aunque falle el hardware.

- **Problemas de Concurrency (Si falla el aislamiento):**

- **Lectura Sucia (Dirty Read):** Leer datos no confirmados
- **Lectura No Repetible:** Un mismo SELECT da datos distintos.
- **Lectura Fantasma:** Aparecen filas nuevas al repetir un SELECT.

(Estos problemas se pueden controlar mediante bloqueos (locks) y niveles de aislamiento (READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ y SERIALIZABLE)).

A mayor aislamiento, más seguridad frente a anomalías pero mayor impacto sobre el rendimiento).

Comandos de Control en MySQL

- **START TRANSACTION (o BEGIN):** Marca el inicio y desactiva autocommit.
- **COMMIT:** Confirma los cambios, haciéndolos permanentes.
- **ROLLBACK:** Deshace los cambios
- **SAVEPOINT nombre:** Crea un punto de control intermedio. Permite hacer un ROLLBACK TO SAVEPOINT.

DCL

El DCL gestiona la seguridad, los permisos, los roles y los usuarios del sistema

Usuario: una persona o una aplicación. Actúa con un determinado conjunto de permisos.

Privilegio: permiso para realizar una acción

Rol: conjunto de privilegios bajo una denominación que puede ser asignado a varios usuarios.

RBAC (Control de Acceso Basado en Roles): Simplifica la seguridad. En lugar de dar permisos persona a persona, se dan a un rol y se asigna la persona al rol.

- **Creación de Roles:** CREATE ROLE 'rol_medico'; Crea el grupo vacío.
- **Vincular:** GRANT 'rol_medico' TO 'medico1'@'%'; El usuario hereda las reglas del rol.

Comandos principales:

- **CREATE USER:** Crear usuarios en el sistema. (Ej. *CREATE USER 'medico1'@'%' IDENTIFIED BY 'Medico123!';*)
- **GRANT:** Otorgar permisos o roles. (Ej. *GRANT SELECT ON clinica_seguridad.pacientes TO 'rol_medico';*)

- **REVOKE:** Retirar permisos.
- **SHOW GRANTS:** Ver permisos actuales.

Optimización de consultas

Es reducir el coste de ejecución de una consulta manteniendo el mismo resultado. El "coste" incluye el tiempo de ejecución, la memoria usada, la carga de la CPU y el número de operaciones.

Los motores de BBDD crean automáticamente índices sobre las PK y las FK, pero **es responsabilidad del profesional forzar al motor de la BD a elegir la ruta óptima** definiendo índices adicionales **en las columnas que más se utilizan para filtrar o para unir.**

Sintaxis: CREATE INDEX nombre_indice ON tabla (columna);

El coste oculto: Aunque aceleran las consultas, ocupan más espacio en disco y hacen que **la escritura sea más lenta**. Por tanto, tener *demasiados* índices ralentizará enormemente las operaciones de escritura.

El comando **EXPLAIN** (o EXPLAIN ANALYZE según el motor de la BD) revela cómo el motor de la BD planea ejecutar una consulta, indicando qué tablas va a recorrer, q tipo de unión usará, y si aprovechará un índice o realizará un escaneo completo permitiendo **identificar los cuellos de botella.**

Reglas de Oro para Optimizar Consultas

1. **Evitar SELECT *:** Nunca traigas datos que no vas a usar. Selecciona solo las columnas necesarias
2. **Usar índices correctamente:** solo en las columnas que se filtran o relacionan mucho.
3. **Optimizar el WHERE:** mediante rangos
4. **Optimizar los JOIN:** Todo campo que utilices en una JOIN debería estar indexado
5. **Limitar resultados:** Usar LIMIT.

SEGURIDAD EN BBDD (auditoría, backups, restauración y protecc. de datos)

El enfoque de “SEGURIDAD EN PROFUNDIDAD” se basa en anticipar riesgos, monitorear acciones, crear defensas múltiples y planificar cómo recuperarte si algo falla.

1ª capa: Auditoría

Mantener un sistema que permita reconstruir lo ocurrido, detectar comportamientos anómalos y demostrar el cumplimiento legal de normativas.

2ª capa: Resiliencia

Depende de una buena estrategia de backups y restauración.

Los **tipos de backup** más comunes son:

- **Completo.**
- **Diferencial** -> Copia los datos desde el último backup completo.
- **Incremental** -> Copia sólo los cambios desde el último backup de cualquier tipo.

La restauración se mide con dos **métricas**:

- **RPO** (Recovery Point Objective) -> Cuántos datos puedes perder medidos en tiempo.
- **RTO** (rec Time ob) -> Cuánto tiempo se tarda en recuperar el sistema.

3ª capa: Protección de datos

Protege incluso en caso de fuga o ataque.

- **Encriptación en reposo** -> Archivos cifrados con TDE.
- **Encriptación en tránsito** -> Conexión cifrada con TLS.
- **Anonimización** -> se eliminan permanentemente los datos que identifiquen a la persona,
- **Pseudoanonimización** -> En sistemas internos confidenciales, Hospitales, Bancos; se sustituyen los datos identificativos (ej.

un DNI) por un ID, guardando la relación real en una tabla secreta separada.

- **Data masking** -> se oculta la información de forma visual (**** 4567).